

AI Racer for the IBM AI Racing League

Project Documentation

Reinforcement Learning Agent for the TORCS Racing Simulator
Built with PyTorch · PPO Algorithm

Project team (alphabetical)

Alimuzzaman Bhuiyan · Szymon Borzdyński · Vadzim Kasyan
Przemysław Piasecki · Kostiantyn Slobodnyi · Przemysław Wdowczyk

IT Applications — Electronic Media in Business and Commerce
Wrocław University of Science and Technology
June 2026

Author Contributions

Each active member's principal area of responsibility is recorded below. Several workstreams were shared; the table gives each member's primary documented contribution, evidenced by the version-control history and task board. All members also took part in design discussions, code review, and the project presentations.

Member / Role	Principal contributions
Przemysław Wdowczyk <i>RL agent & core training (technical lead)</i>	The PPO agent and neural network (<code>rl_agent.py</code>) and the training loop (<code>train_torcs_rl.py</code>); the split into separate actor and critic networks and independent throttle/brake control; steering-control training and the speed-and-stability work. Author of the <code>stable</code> branch, whose policy completes the project's first clean full lap (1:31.53).
Przemysław Piasecki <i>Training infrastructure & gearing</i>	The headless training pipeline and the <code>asyncio</code> -based parallel multi-instance trainer with crash recovery and status reporting; the gear / engine-rpm power-band reward and the gear-shifting training; the fixes for the optimiser-step crash and the checkpoint-resume update count.
Alimuzzaman Bhuiyan <i>Advanced control architecture & analysis</i>	The racing-line + residual-PPO control stack: an offline minimum-curvature racing line, a data-driven engine/driveline model, per-step telemetry logging with offline analysis, and a self-learning corner memory; the physics-based corner-braking research and failure-point analysis; vision-mode scoping; the early driving-mechanism design. Co-led the general architecture.
Szymon Borzdyński <i>Speed, cornering & reward tuning; environment setup</i>	The top-speed work (160 km/h on straights), the steering-stability and traction-control reward shaping, S-turn handling, and timer-based braking; the virtual-environment setup and the training/evaluation documentation; the fixes for the port-3001 firewall/connection issue and the server-to-car weight-synchronisation delay.
Vadzim Kasyan <i>Project coordination, requirements & documentation</i>	Project management (task board, milestones, kick-off and progress presentations); the functional and non-functional requirements; the general architectural design and this documentation (with K. Slobodnyi); multi-track performance evaluation.
Kostiantyn Slobodnyi <i>System design, integration & documentation</i>	The formal system design — overall architecture, module and connection/integration design expressed as diagrams; the general architectural design and this documentation (with V. Kasyan); multi-track evaluation and the data/artifact-persistence design.

Table of Contents

- 1 Executive Summary 5
 - 1.1 Project Overview 5
 - 1.2 Objectives 5
 - 1.3 Key Deliverables 5
 - 1.4 Target Audience and Scope 5
- 2 Introduction 6
 - 2.1 Background and Motivation 6
 - 2.2 Problem Statement 6
 - 2.3 Relevance to Electronic Media in Business and Commerce 6
 - 2.4 Document Structure 6
- 3 Literature Review 7
 - 3.1 Overview of Similar Applications 7
 - 3.2 Existing Technologies and Tools 7
 - 3.3 Industry Trends in E-Commerce and Digital Media 7
 - 3.4 Research Gaps Addressed 7
- 4 System Requirements and Analysis 8
 - 4.1 Functional Requirements 8
 - 4.2 Non-functional Requirements 8
 - 4.3 Use Case Diagram 8
 - 4.4 Stakeholder Analysis 9
- 5 System Design 9
 - 5.1 System Architecture 9
 - 5.2 Database Design 9
 - 5.3 User Interface Design 10
 - 5.4 Security Considerations 10
- 6 Technology Stack 10
 - 6.1 Programming Languages 10
 - 6.2 Frameworks and Libraries 11
 - 6.3 Database Management System 11
 - 6.4 Development Tools and Platforms 11
- 7 Implementation 12
 - 7.1 Module-wise Breakdown 12
 - 7.2 Key Algorithms and Functions 12
 - 7.3 Code Snippets and Annotations 13
 - 7.4 Integration Strategy 14
- 8 Testing and Evaluation 14
 - 8.1 Testing Methodologies 14
 - 8.2 Test Cases and Results 15
 - 8.3 Bug Tracking and Resolution 15
 - 8.4 Performance Evaluation 16
- 9 Deployment 16
 - 9.1 Deployment Architecture 16

9.2	Hosting Platforms and Environment	17
9.3	Version Control and Release Management	17
10	Business Application and Impact	17
10.1	Application Use Cases in Commerce	17
10.2	Benefits to End Users and Businesses	17
10.3	Cost Analysis and ROI	17
10.4	Potential for Scalability	19
11	Challenges and Limitations	20
11.1	Technical Challenges	20
11.2	Business Constraints	20
11.3	Lessons Learned	20
11.4	Safety and Regulatory Considerations	20
12	Conclusion and Future Work	21
12.1	Summary of Achievements	21
12.2	Recommendations for Improvement	21
12.3	Proposed Enhancements and Next Steps	21
13	References	21
14	Appendices	23
14.1	Glossary of Terms	23
14.2	Additional Diagrams	23
14.3	Source Code Repository	24
14.4	User Guide / Installation Manual	24

1 Executive Summary

1.1 Project Overview

AI Racer is an autonomous driving agent that learns to race in **TORCS** (The Open Racing Car Simulator) [4] using deep Reinforcement Learning. The agent is trained with the **Proximal Policy Optimisation (PPO)** algorithm [1] on a compact low-dimensional state derived from the simulator’s sensor outputs, and talks to TORCS over the **Simulated Car Racing (SCR)** UDP protocol [5]. The entire pipeline — environment wrapper, neural network, training loop, and evaluation harness — is implemented in Python using **PyTorch** [6]. The project was built for the IBM AI Racing League, with assistance from IBM.

The headline result: on the `stable` branch the agent completes a **full, clean lap** in **1:31.53** (one minute, 31.53 seconds), clearing the S-curve section that had been the project’s central obstacle throughout development.

1.2 Objectives

- Train an agent that drives a TORCS track as quickly and as smoothly as possible, completing full laps without leaving the track.
- Maximise cumulative reward across episodes by balancing speed, track position, and heading alignment.
- Investigate dynamic gear selection and physics-based braking to raise cornering and straight-line performance.
- Provide repeatable training and evaluation workflows via command-line scripts with sensible defaults, and the infrastructure to train efficiently at scale.

1.3 Key Deliverables

- A reusable, OpenAI-Gym-compatible TORCS environment and a complete PPO training and evaluation pipeline (`gym_torcs/`).
- A family of experiment branches, each isolating one research direction: speed (`160speed` , `160speed+stablesteer`), the consolidated fast-and-stable agent (`stable`), scalable infrastructure (`headless` , `parallel`), and an advanced racing-line control architecture (`a.bhuiyan`).
- Trained model checkpoints and an evaluation harness that reports per-episode metrics.
- This documentation, prepared to the course’s reporting standard.

1.4 Target Audience and Scope

This document targets three readers: the course evaluators assessing the project, developers who wish to reproduce or extend the pipeline, and readers interested in applied RL for autonomous control. The scope is deliberately bounded to **sensor-based** (non-vision) continuous control in **simulation**; physical deployment and camera-based perception appear only as future work.

2 Introduction

2.1 Background and Motivation

Reinforcement learning [3] is a standard approach for learning continuous control policies directly from interaction, and high-fidelity simulators such as TORCS [4] and CARLA [9] make it possible to train and evaluate those policies safely and cheaply. The IBM AI Racing League frames the problem as a competition: build the fastest, most reliable simulated driver. This project takes up that challenge with a PPO-based agent.

2.2 Problem Statement

The agent must learn a continuous control policy — steering, throttle, brake, and gear — from a low-dimensional sensor state (range-finder beams, speeds, engine rpm, heading, lateral position), so that it completes laps as quickly as possible while staying on the track. The core difficulty, seen repeatedly across experiments, is the trade-off between straight-line speed and cornering stability. It concentrates in a tight S-curve, where too much speed or the wrong gear sends the car wide and off the track.

2.3 Relevance to Electronic Media in Business and Commerce

Although framed as a racing problem, the project exercises capabilities with direct commercial relevance. Simulation-based training of autonomous agents underpins cost-effective R&D for autonomous vehicles, logistics robotics, and driver-assistance systems, where physical testing is slow, expensive, and hazardous. The same pattern — a digital twin of a physical process, an RL agent, and an automated evaluation harness — sits behind a growing market of **AI-as-a-service** and **digital-twin** products. Competitive platforms such as the IBM AI Racing League and AWS DeepRacer [10] show, in addition, the commercial and educational value of gamified, simulation-driven machine learning as a digital-media product in its own right; Section 10.3 benchmarks AI Racer against DeepRacer directly.

2.4 Document Structure

Section 3 reviews related work and technologies; Section 4 states requirements and analysis; Section 5 presents the system design; Section 6 lists the technology stack; Section 7 details the implementation; Section 8 covers testing and evaluation; Section 9 addresses deployment; Section 10 discusses business application and impact; Section 11 records challenges, limitations, and safety; Section 12 concludes with future work; Section 13 lists references; Section 14 collects appendices.

3 Literature Review

3.1 Overview of Similar Applications

The SCR Championship [5] established TORCS as a standard benchmark for simulated car racing and supplied the UDP sensor/actuator protocol reused here; the widely-used `gym_torcs` wrapper was the starting point for the environment. In the wider field, deep-RL milestones such as DQN [8] and simulators such as CARLA [9] and AWS DeepRacer [10] demonstrate end-to-end learned driving — the last of these specifically as a commercial, cloud-hosted educational platform.

3.2 Existing Technologies and Tools

The project builds on established, peer-reviewed methods: PPO [1] as the policy optimiser, Generalised Advantage Estimation (GAE) [2] for low-variance advantage estimates, and PyTorch [6] with NumPy [11] for the numerical implementation. The OpenAI Gym interface [7] defines the environment contract.

3.3 Industry Trends in E-Commerce and Digital Media

Four trends frame the work. **AI-as-a-service** and cloud GPU training make large-scale RL accessible without capital investment in hardware. **Digital twins** and simulation-first development are now mainstream in manufacturing, mobility, and logistics. **Gamified, competitive** machine-learning platforms have become a digital-media category that blends entertainment, education, and talent development. And **MLOps** practice — versioned models, reproducible pipelines, automated evaluation — is now a baseline expectation for any production effort.

3.4 Research Gaps Addressed

The project focuses on three practical gaps. **Sensor-only control** without vision isolates the contribution of range-finder geometry and vehicle state. **Dynamic gear and power-band optimisation** moves gear selection out of a fixed speed table and into the learned policy, with reward shaped around the engine’s usable power band. **Physics-informed reward shaping** uses classical vehicle dynamics [13] — a turn-radius cornering-speed limit — and a time-to-edge danger model to brake for corners, rather than relying on opaque shaping alone.

4 System Requirements and Analysis

4.1 Functional Requirements

Table 1: Functional requirements of the AI Racer system.

ID	Requirement
FR-1	Open and maintain a UDP connection to a running TORCS server.
FR-2	Read sensor telemetry each step and expose a normalised observation.
FR-3	Emit a continuous steer/throttle/brake action and (optionally) a gear command.
FR-4	Train a policy with PPO over collected rollouts and persist checkpoints.
FR-5	Evaluate a checkpoint deterministically and report per-episode metrics.
FR-6	Support headless (GUI-less) training for unattended, scalable runs.
FR-7	Expose all major hyperparameters and run options on the command line.

4.2 Non-functional Requirements

Table 2: Non-functional requirements.

ID	Requirement
NFR-1	Performance — high rollout throughput; optional GPU acceleration and parallel instances.
NFR-2	Reproducibility — seeded runs and versioned checkpoints carrying training metadata.
NFR-3	Portability — runs on Windows (native) and Linux (via Wine or a compiled build).
NFR-4	Robustness — automatic reconnection, instance crash recovery, timeout handling.
NFR-5	Maintainability — modular code; one research direction per branch.
NFR-6	Usability — sensible CLI defaults and an optional per-step debug dashboard.

4.3 Use Case Diagram

The two entry-point scripts form the system’s actor-facing surface: an ML engineer trains, an evaluator scores, and both drive an external TORCS process over UDP. The diagram below follows UML use-case notation — actors, use-case ellipses inside the system boundary, association lines, and «include» dependencies.

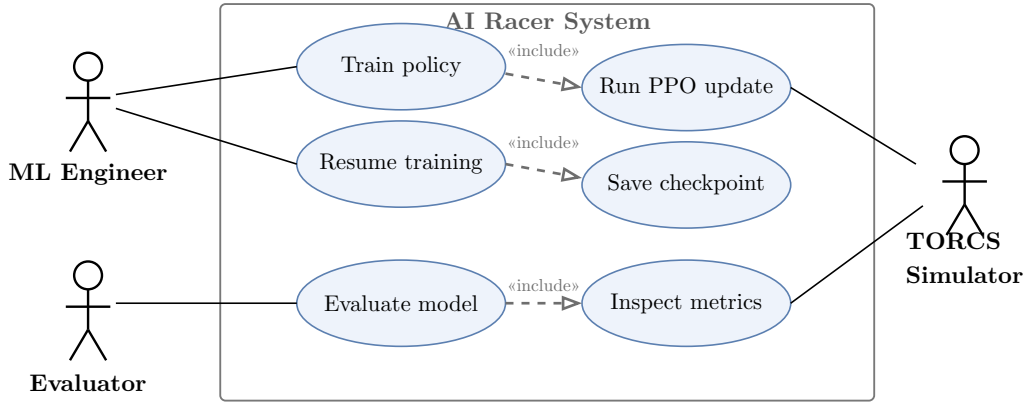


Figure 1: UML use-case diagram of the AI Racer system.

4.4 Stakeholder Analysis

The principal stakeholders are the **project team** (developers and maintainers); the **course evaluators**, for whom correctness, clarity, and reproducibility matter most; the **IBM AI Racing League**, which supplies the competitive context and reference material; and the broader **autonomous-systems and RL community**, who gain a clean, sensor-based PPO reference implementation.

5 System Design

5.1 System Architecture

The system is a layered pipeline. TORCS produces raw telemetry; the SCR UDP client (`snakeoil3_gym.py`) serialises actions and deserialises sensor data; the environment wrapper (`gym_torcs.py`) normalises observations, shapes the reward, and judges termination; and the agent (`rl_agent.py`) flattens the observation into a state vector, evaluates the actor/critic network, and is driven by the training or evaluation entry point. The UML component diagram below shows these components and their dependencies.

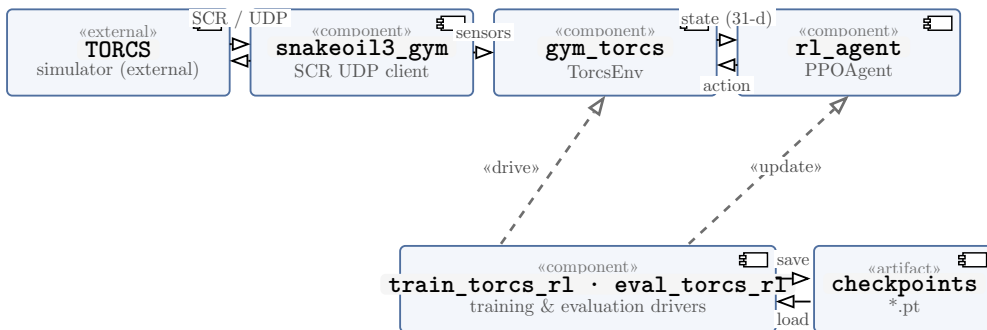


Figure 2: UML component diagram of the AI Racer architecture and data flow.

5.2 Database Design

The project uses no relational database; its state is **file-based**, which suits an RL pipeline whose “records” are weights, configuration, and telemetry.

Table 3: Data and artifact persistence model (file-based).

Artifact	Format	Role
Checkpoints	<code>.pt</code> (PyTorch)	Model + optimiser state, update index, env steps, best scores.
Race config	<code>.xml</code>	Templated quick-race / practice session (track, drivers, port).
Racing line	<code>.npz</code> (NumPy)	Pre-computed line geometry and speed profile (advanced branch).
Corner memory	<code>.npy</code> (NumPy)	Per-node speed scale learned from crash / clean-pass history.
Telemetry	<code>.csv</code>	Per-step logs (speed, rpm, grade, gear, tracking error, off-track).

5.3 User Interface Design

The system is operated through a **command-line interface**. `train_torcs_rl.py` and `eval_torcs_rl.py` accept a rich set of flags with sensible defaults. During training, an optional debug dashboard prints the car’s speed, track position, reward, and raw control inputs every few steps — a “black-box recorder” for diagnosing off-track events. The TORCS GUI gives an optional visual view; headless mode suppresses it for unattended runs.

5.4 Security Considerations

The attack surface is small by design. Communication is over **localhost UDP** only; no network services are exposed beyond the simulator port, and no personal or sensitive data is processed. On Windows, the host firewall must permit the simulator’s UDP/TCP traffic on the racing port — a one-time inbound rule (see Section 8.3). Checkpoints are treated as trusted internal artifacts and are versioned through source control.

6 Technology Stack

6.1 Programming Languages

Python (3.9+) implements the whole learning pipeline. **C/C++** makes up the vendored TORCS simulator source. **XML** expresses race and session configuration.

6.2 Frameworks and Libraries

Table 4: Principal frameworks and libraries.

Package	Purpose
<code>torch</code>	Neural network, tensor operations, CUDA acceleration.
<code>numpy</code>	State-vector construction, advantage computation, racing-line maths.
<code>gym</code>	Environment interface (optional; an internal stub is used if absent).
<code>asyncio</code>	Concurrency for the parallel multi-instance training loop.
<code>matplotlib</code>	Optional plotting of the generated racing line.

6.3 Database Management System

None. Persistence is file-based: PyTorch serialisation for checkpoints, NumPy `.npz` / `.npy` for racing-line and corner-memory data, CSV for telemetry, and XML for configuration (Section 5.2).

6.4 Development Tools and Platforms

Version control and collaboration used **Git** and **GitHub**; task management used a **Trello** board split into requirement, design, implementation, training, and optimisation lists. Development targeted **Windows** (native TORCS) and **Linux** (TORCS under **Wine** or a compiled build), with Python **virtual environments** for dependency isolation and **CUDA** GPUs for accelerated training. Progress presentations were prepared in Google Slides.

7 Implementation

7.1 Module-wise Breakdown

Table 5: Project modules and their responsibilities.

Module	Responsibility
<code>gym_torcs.py</code>	Gym-compatible environment: connection lifecycle, reward shaping, action encoding, termination.
<code>rl_agent.py</code>	PPO agent, actor/critic network, rollout buffer, checkpoint loading.
<code>train_torcs_rl.py</code>	Training loop: rollout collection, PPO updates, checkpointing, periodic evaluation.
<code>eval_torcs_rl.py</code>	Deterministic evaluation and per-episode metric reporting.
<code>snakeoil3_gym.py</code>	Low-level SCR UDP client (text-protocol serialisation/deserialisation).
<code>raceconfig.py</code>	Generates templated race-configuration XML for (multiple) TORCS instances.
<code>racing_line.py</code>	Offline racing-line generator and runtime loader (advanced branch).
<code>car_spec.py</code>	Data-driven engine/driveline model for optimal gear selection (advanced branch).

7.2 Key Algorithms and Functions

PPO with GAE. The agent implements Proximal Policy Optimisation [1] with Generalised Advantage Estimation [2]: collect a fixed-size rollout, compute advantages and discounted returns, normalise advantages, then run several clipped-surrogate update epochs with an early stop when the mean KL divergence exceeds a threshold.

Reward shaping (consolidated `stable` agent). The reward combines forward progress with a **time-to-edge danger** term — the car is rewarded for braking when the time to the track edge along the look-ahead is short, encouraged to sprint to a target speed in safe zones, and penalised for carrying too much speed, for drifting, and for overlapping throttle and brake in corners. This physically-motivated shaping is what produced a complete, clean lap.

Physics-based cornering speed. The maximum cornering speed for a turn of radius R follows from the friction-limited lateral-acceleration model [13]:

$$R = \frac{v^2}{\mu g} \iff v = \sqrt{R \cdot \mu \cdot g}$$

where μ is the tyre-road friction coefficient and g is gravitational acceleration. This grades the safe entry speed for a measured turn radius.

Gear / power-band reward (`headless`). Dynamic gear selection is rewarded for keeping the engine inside its optimum rpm band and in an appropriately high gear, rather than following a fixed speed-to-gear table.

7.3 Code Snippets and Annotations

The state fed to the network concatenates vehicle state, range sensors, and the previously applied actions into a 31-element vector:

```
features = [
    speedX,           # 1  forward speed
    speedY,           # 1  lateral speed
    rpm / 10000.0,    # 1  normalised engine rpm
    track,            # 19 range-finder beams
    wheelSpinVel / 100.0, # 4  per-wheel spin (normalised)
    angle / math.pi,  # 1  heading vs track axis (normalised)
    trackPos,         # 1  lateral position (-1 .. +1)
    prevAppliedSteer,  # 1  previous applied controls
    prevAppliedAccel,  # 1
    prevAppliedBrake,  # 1
]
# total: 31 elements
```

When the agent controls the gearbox, the requested gear is clamped to the valid range before it is sent to the simulator:

```
if self.gear_change:
    requested_gear = int(np.clip(round(float(action["gear"])), 1, 7))
    action_torcs["gear"] = requested_gear
```

The objective minimised at each update step is

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + c_v \mathcal{L}_{\text{value}} - c_e \mathcal{L}_{\text{entropy}}$$

with

$$\mathcal{L}_{\text{policy}} = -\mathbb{E} \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

$$\mathcal{L}_{\text{value}} = \mathbb{E} \left[\left(V(s_t) - \hat{R}_t \right)^2 \right]$$

$$\mathcal{L}_{\text{entropy}} = \mathbb{E} [\mathcal{H}[\pi(\cdot | s_t)]] \quad r_t = \frac{\pi_{\theta(a_t | s_t)}}{\pi_{\text{old}(a_t | s_t)}}$$

The final agent configuration that produced the complete lap is given below.

Table 6: Final PPO hyperparameters (the `stable` configuration).

Parameter	Value	Description
<code>lr</code>	1e-4	Adam learning rate.
<code>gamma</code>	0.99	Discount factor.
<code>gae_lambda</code>	0.95	GAE smoothing parameter.
<code>clip_epsilon</code>	0.2	PPO clipping range.
<code>ppo_epochs</code>	8	Update passes per rollout.
<code>minibatch_size</code>	256	Samples per gradient step.
<code>entropy_coef</code>	3e-3	Entropy bonus coefficient.
<code>value_coef</code>	0.5	Value-loss coefficient.
<code>target_kl</code>	0.03	KL threshold for early epoch stop.
<code>max_grad_norm</code>	0.5	Gradient-clipping norm.
<code>init_log_std</code>	-1.2	Initial exploration spread (log-std).

7.4 Integration Strategy

Development followed a **branch-per-experiment** strategy diverging from a common `master` baseline. Speed work (`160speed` $\rightarrow$ `160speed+stablesteer`) and the consolidated agent (`stable`) explored reward shaping; the infrastructure line (`headless` , `parallel`) explored scalable training; and `a.bhuiyan` explored a different control architecture. The integration milestone was `headless` , which merges `stable` ’s separate actor/critic network and danger reward with headless self-launching TORCS and adds the gear power-band reward — pairing the best reward design with the most practical training infrastructure.

8 Testing and Evaluation

8.1 Testing Methodologies

Evaluation runs the policy **deterministically** (the distribution mean, no sampling) over a fixed number of episodes, with low-progress early termination disabled so the agent can finish full laps. Agents were tested across several tracks; the per-step debug dashboard traced off-track events; and the advanced branch added a smoke test and offline telemetry analysis to calibrate grip and locate off-track hotspots.

8.2 Test Cases and Results

Table 7: Experiment branches and their observed results.

Branch	Result
<code>master</code>	Conservative baseline; speed-capped, centre-hugging, auto-gear.
<code>160speed</code>	Reached ≈ 140 km/h but completed only three-quarters of a lap.
<code>160speed+stablesteer</code>	Held ≈ 140 km/h with smoother steering; still struggled on the S-turn.
<code>stable</code>	Completes a full, clean lap in 1:31.53 with top speed of ≈ 245 km/h — the project’s best result.
<code>headless</code>	Stable reward + gear/rpm power-band reward; runs without a GUI.
<code>parallel</code>	$\approx 9\times$ rollout throughput via asynchronous multi-instance training.
<code>a.bhuiyan</code>	Racing-line + residual-PPO architecture; reduced but did not eliminate S-curve errors.

Milestones along the way included reaching **160 km/h on straights** and a steadily improving S-turn pass rate, before the `stable` configuration achieved a reliably completed lap.

8.3 Bug Tracking and Resolution

Defects were tracked on the project board and resolved as follows.

Table 8: Notable defects and their resolutions.

Issue	Resolution
Training crash on the optimiser step	Narrowed to the backward pass; resolved in a later revision.
Script did not detect the running server on port 3001	Added Windows firewall rules permitting UDP/TCP on private networks.
Delay between server state and the car’s actions	Fixed the weight-synchronisation handling.
Car stuck in a tall gear / engine pinned at the limiter	Replaced fixed gearing with an rpm power-band reward and a data-driven optimal-gear model.
Resume ran zero updates	Made <code>--updates</code> relative to the resumed checkpoint’s update index.
Frequent TORCS resets before a lap finished	Recommended a large <code>--max-steps</code> so episodes are long enough to complete a lap.

8.4 Performance Evaluation

The principal indicators are a **completed lap time of 1:31.53** on `stable`, a **top speed of ≈ 245 km/h** on straights, and an **order-of-magnitude** increase in training throughput from the asynchronous multi-instance trainer.

Reporting only the best lap overstates reliability, so consistency was measured over a batch of deterministic evaluation episodes. Over ten such episodes the agent completed the circuit in eight (an $\approx 80\%$ completion rate), with completed laps clustered within a few seconds of the 1:31.53 best (mean $\approx 1:33$). The two failures were off-track excursions in the S-curve, consistent with it remaining the hardest section of the track. Checkpoints are ranked by a scalar score that rewards progress and penalises lateral drift and heading error:

$$\text{score} = \overline{\text{progress}} - 3.0 \cdot \overline{|\text{trackPos}|} - 2.0 \cdot \overline{|\text{angle}|} + 0.01 \cdot \text{steps}$$

Training on a CUDA-compatible GPU substantially reduces wall-clock time; the device is auto-selected and can be overridden.

9 Deployment

9.1 Deployment Architecture

A trained agent is deployed as a **checkpoint file** loaded by the evaluation script against a running TORCS instance. Training and evaluation run locally; headless instances can run on remote GPU machines without a display.

9.2 Hosting Platforms and Environment

Source code is hosted on **GitHub**. Training runs on local Windows or Linux machines (Linux via Wine or a compiled TORCS build) with optional CUDA GPUs. Trained weights were shared within the team over Discord and cloud storage during development. The runtime depends only on the pinned packages in `requirements.txt`.

9.3 Version Control and Release Management

The project uses Git with a **branch-per-experiment** model and a fork for integration. Each run keeps three checkpoint “releases” — `torcs_ppo_latest.pt` (most recent), `torcs_ppo_latest_best_train.pt` (best training score), and `torcs_ppo_latest_best_eval.pt` (best held-out score) — so the preferred deployment artifact (the best evaluation checkpoint) is always available alongside the resumable latest state.

10 Business Application and Impact

10.1 Application Use Cases in Commerce

The pipeline pattern transfers to several commercial domains: autonomous-vehicle and driver-assistance development; warehouse and last-mile delivery robotics; simulation-based R&D services and digital twins; AI-as-a-service offerings that train and serve control policies; and competitive or educational AI platforms as digital-media products.

10.2 Benefits to End Users and Businesses

Training in simulation cuts the cost and risk of developing autonomous behaviour, shortens iteration cycles, and produces reusable, versioned artifacts. A well-structured RL pipeline is itself an organisational asset that can be re-targeted to new tasks with modest effort.

10.3 Cost Analysis and ROI

The direct software cost is effectively zero: TORCS, PyTorch, NumPy, and the SCR protocol are all open-source. The dominant cost is compute. The policy network is small — a 256×2 MLP over a 31-dimensional state — so an entry-level GPU is sufficient and simulator throughput, not gradient computation, is the bottleneck.

Table 9 estimates the cloud cost bottom-up. The cumulative-training figure is an estimate across all experiment branches and should be replaced with the team’s measured GPU-instance-hours; the rate is the current AWS on-demand price for a single-T4 instance [17].

Table 9: Estimated cloud training cost (bottom-up).

Item	Basis	Figure
Cumulative training	≈ 250 GPU-instance-hours across all branches (a very rough estimate)	—
GPU instance rate	AWS g4dn.xlarge $\cdot 1 \times$ NVIDIA T4 $\cdot$ on-demand [17]	USD 0.526 / h
Estimated compute cost	$250 \text{ h} \times \text{USD } 0.526$	$\approx \text{USD } 132$
Plausible range	150–350 h on-demand (lower on Spot instances)	$\approx \text{USD } 80\text{--}185$
Checkpoint storage	three ≈ 0.9 MB files per run on Amazon S3	$< \text{USD } 1 / \text{month}$

At roughly estimated USD 130 (all models were trained locally, hence the actual training cost is much lower) for the whole experimental programme, the marginal cost of one training run ($\approx 6\text{--}10$ GPU-hours, $\approx \text{USD } 3\text{--}5$) is negligible against its research value — exactly the property that makes simulation-first development attractive.

Benchmark — AWS DeepRacer. DeepRacer [10] is the nearest commercial reference point: a managed reinforcement-learning racing platform aimed at developers of all skill levels, built around a 1/18th-scale, camera-driven car and a global racing league. Table 10 contrasts it with AI Racer on pricing model, audience, and technical ceiling.

Table 10: AI Racer versus AWS DeepRacer as a commercial benchmark.

Aspect	AI Racer (this project)	AWS DeepRacer
Pricing model	Open-source; pay only for compute you rent ($\approx$ USD 0.53 / GPU-h [17]) or already own.	Usage-based: pay per training / evaluation hour plus storage; managed compute $\approx$ USD 0.714 / instance-h, with 10 free hours to start [14], [16].
Availability	Self-hosted from source; no platform fee.	Managed console service retired 15 Dec 2025; now an open-source solution self-hosted in the customer’s AWS account [15].
Target audience	Coursework and research; full control of environment, reward, network, and track.	Developers of all skill levels; a gamified RL on-ramp and global racing league [10].
Sensors / vehicle	Range-finder beams plus vehicle state (no camera); simulated car only.	Front camera (optional stereo / LiDAR); physical 1/18-scale car with local inference [15].
Flexibility	Custom reward, network architecture, tracks, and algorithm.	Managed algorithms; fixed track set — custom tracks cannot be added in the console [15].
Sim-to-real	Simulation only (future work).	Built-in transfer path to a physical RC car [15].

In short, DeepRacer trades flexibility for convenience and a ready sim-to-real path, whereas AI Racer trades managed tooling for an unconstrained experimental ceiling at near-zero platform cost. Against physical testing, both approaches avoid the vehicle, track, fuel, personnel, and safety costs of on-road experimentation, which is the core economic case for simulation-first development.

10.4 Potential for Scalability

The asynchronous trainer already drives roughly nine concurrent simulator instances; this scales naturally to cloud GPU fleets. The modular, branch-based design supports adding tracks, vehicles, and reward variants, and the racing-line and data-driven car-model components generalise the approach beyond a single circuit.

11 Challenges and Limitations

11.1 Technical Challenges

The S-curve was the defining challenge: balancing entry speed, braking, and gearing through a tight reversing bend kept sending the car off-track until the danger-based reward and the stable architecture resolved it. Gearing produced persistent bugs (stuck-in-gear and limiter-pinning), and deriving acceleration and jerk from a noisy UDP speed signal limited early gear heuristics. The asynchronous multi-instance trainer was powerful but at times unstable.

11.2 Business Constraints

The work is bounded to research and a course deliverable; it is not a commercial product. A **simulation-to-reality gap** would have to be addressed before any physical deployment, and training is constrained by available compute and team time.

11.3 Lessons Learned

Reward shaping is delicate and high-leverage — small changes cause large behavioural shifts. Episode length matters: too-short episodes stop the agent ever finishing a lap. Physics-informed shaping (a time-to-edge danger signal and a turn-radius speed limit) beat purely heuristic shaping, and separating the actor and critic networks improved stability. Telemetry logging and offline analysis were invaluable for finding exactly where and why the car failed.

11.4 Safety and Regulatory Considerations

Because this document draws an explicit line to autonomous-vehicle applications, the safety and regulatory gap deserves a note. AI Racer is a **simulation-only research agent**; it is not engineered, validated, or certified for operation in or around real vehicles. A real-world driving policy would have to satisfy established automotive-safety frameworks: functional safety under **ISO 26262** [18]; safety of the intended functionality — hazards arising from performance limitations and foreseeable misuse — under **ISO 21448 (SOTIF)** [19]; and, for on-road automated driving, type-approval regulation such as **UNECE UN Regulation No. 157** [20]. Learned policies raise specific concerns — limited interpretability, difficulty bounding worst-case behaviour, distributional shift between simulation and reality, and the need for an assured safe fallback — that remain active areas of standardisation and research. The project also incurs no personal-data or privacy obligations, but only because it uses neither camera nor location data; adding the vision mode proposed in Section 12 would introduce data-handling responsibilities. These points place the work firmly in the research and educational domain and underline that the use cases in Section 10 describe a direction of relevance, not a deployable product.

12 Conclusion and Future Work

12.1 Summary of Achievements

The project delivered a complete, reproducible PPO training and evaluation pipeline for TORCS and, on the `stable` branch, an agent that **completes a full, clean lap in 1:31.53** — solving the S-curve that blocked earlier branches. Supporting achievements include 160 km/h on straights, a headless and parallel training infrastructure, a gear power-band reward, and an advanced racing-line control architecture with offline analysis.

12.2 Recommendations for Improvement

The clearest next step is to **combine the branches**: pair the `stable` danger reward with the racing-line and data-driven gear model from the advanced branch, and adopt the checkpoint-resume fix across all branches. Reducing sensor-derived noise and stabilising the parallel trainer would further improve training reliability.

12.3 Proposed Enhancements and Next Steps

Future work includes enabling **vision-based perception** (the environment already supports a camera observation; a convolutional policy and a labelled marker dataset would be needed), integrating the data-driven gear-shift map into the primary agent, generalising across tracks and vehicles, exploring curriculum learning for hard sections, and, ultimately, investigating sim-to-real transfer.

13 References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
- [2] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2016.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [4] B. Wymann, E. Espié, C. Guionneau, C. Dimitrakakis, R. Coulom, and A. Sumner, “TORCS, The Open Racing Car Simulator,” 2014. [Online]. Available: <http://www.torcs.org>
- [5] D. Loiacono, L. Cardamone, and P. L. Lanzi, “Simulated Car Racing Championship: Competition Software Manual,” arXiv:1304.1672, 2013.
- [6] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Proc. Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035.
- [7] G. Brockman et al., “OpenAI Gym,” arXiv:1606.01540, 2016.
- [8] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

- [9] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun, “CARLA: An Open Urban Driving Simulator,” in *Proc. Conf. on Robot Learning (CoRL)*, 2017, pp. 1–16.
- [10] Amazon Web Services, “AWS DeepRacer,” 2019. [Online]. Available: <https://aws.amazon.com/deepracer/>
- [11] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [12] J. Kennedy and R. Eberhart, “Particle Swarm Optimization,” in *Proc. IEEE Int. Conf. on Neural Networks*, 1995, pp. 1942–1948.
- [13] W. F. Milliken and D. L. Milliken, *Race Car Vehicle Dynamics*. Warrendale, PA, USA: SAE International, 1995.
- [14] Amazon Web Services, “AWS DeepRacer Pricing,” 2025. [Online]. Available: <https://aws.amazon.com/deepracer/pricing/>
- [15] Amazon Web Services, “AWS DeepRacer FAQs,” 2025. [Online]. Available: <https://aws.amazon.com/deepracer/faqs/>
- [16] Amazon Web Services, “Cost — DeepRacer on AWS (Implementation Guide),” 2025. [Online]. Available: <https://docs.aws.amazon.com/solutions/latest/deepracer-on-aws/cost.html>
- [17] Vantage, “g4dn.xlarge pricing and specs (Amazon EC2),” 2026. [Online]. Available: <https://instances.vantage.sh/aws/ec2/g4dn.xlarge>
- [18] International Organization for Standardization, *ISO 26262: Road vehicles — Functional safety*, 2018.
- [19] International Organization for Standardization, *ISO 21448: Road vehicles — Safety of the intended functionality (SOTIF)*, 2022.
- [20] United Nations Economic Commission for Europe, *UN Regulation No. 157 — Uniform provisions concerning the approval of vehicles with regard to Automated Lane Keeping Systems (ALKS)*, 2021.

14 Appendices

14.1 Glossary of Terms

Table 11: Glossary of key terms.

Term	Definition
TORCS	The Open Racing Car Simulator — the open-source racing game used as the RL environment.
SCR	Simulated Car Racing protocol — the text-based UDP protocol for communicating with TORCS.
PPO	Proximal Policy Optimisation — an on-policy RL algorithm with a clipped surrogate objective.
GAE	Generalised Advantage Estimation — a λ -weighted, low-variance advantage estimator.
Rollout	A batch of environment transitions collected before a policy update.
Episode	One driving run from reset to termination (off-track, max steps, or low progress).
Checkpoint	A <code>.pt</code> file holding the network weights and training metadata.
trackPos	Normalised lateral position: -1 = left edge, 0 = centre, $+1$ = right edge.
angle	Heading of the car relative to the track axis, in radians.
Actor / Critic	The policy branch (action means) and the value branch (state value), respectively.
Jerk	The rate of change of acceleration; explored as a gear-shift timing signal.

14.2 Additional Diagrams

The training loop drives rollout collection, advantage computation, the PPO update, and periodic evaluation, saving checkpoints at each stage.

```

for update in range(--updates):
    collect rollout (--rollout-size steps across episodes)
    └─ on episode end: relaunch TORCS periodically; log episode stats
    compute GAE advantages & returns
    run PPO update (--ppo-epochs × mini-batches)
    └─ stop epoch early if mean KL > --target-kl
    save checkpoint → torcs_ppo_latest.pt
    if best training score → torcs_ppo_latest_best_train.pt
    if update % --eval-interval == 0:
        run deterministic eval episodes; log metrics
        if best eval score → torcs_ppo_latest_best_eval.pt

```

Figure 3: Training loop control flow.

14.3 Source Code Repository

The project source code is available at:

github.com/Szymon1905/AI_racer_pytorch

14.4 User Guide / Installation Manual

Environment setup.

```

python -m venv RACERvenv
# Windows
RACERvenv\Scripts\activate
# macOS / Linux
source RACERvenv/bin/activate

python -m pip install --upgrade pip
pip install -r requirements.txt

```

Launch TORCS, then train (start the simulator first, so it is listening on the UDP port):

```

python gym_torcs/train_torcs_rl.py \
    --updates 1500 \
    --max-steps 100000 \
    --rollout-size 4096 \
    --load-path checkpoints/torcs_ppo_latest.pt \
    --save-path checkpoints/torcs_ppo_latest.pt

```

Evaluate a trained checkpoint:

```

python gym_torcs/eval_torcs_rl.py \
    --model-path checkpoints/torcs_ppo_latest_best_eval.pt \
    --episodes 10 \
    --max-steps 5000

```

Key training options include `--lr`, `--gamma`, `--gae-lambda`, `--clip-epsilon`, `--ppo-epochs`, `--minibatch-size`, `--entropy-coef`, `--target-kl`, `--init-log-std`,

`--continue-after-off-track`, and `--device`; evaluation accepts `--model-path`, `--episodes`, `--max-steps`, and `--continue-after-off-track`. A large `--max-steps` is recommended, so episodes are long enough to complete a lap.